

LAPORAN TEKNIS & BUKU PANDUAN TERPERINCI (MANUAL GUIDE)

Sistem Enterprise Resource Planning (ERP) Berbasis
Metadata-Driven Architecture & Low-Code Platform

BAB 1: PENDAHULUAN & RINGKASAN EKSEKUTIF

1.1 Latar Belakang & Filosofi Sistem

Dalam pengembangan sistem ERP enterprise tradisional, penambahan atau modifikasi modul (seperti perubahan label kolom, penambahan field baru, atau penyesuaian aturan validasi) seringkali membutuhkan waktu siklus pengembangan yang panjang. Pengembang harus mengubah kode sumber (HTML/Java), melakukan kompilasi ulang, dan mendeploy ulang seluruh aplikasi ke server produksi.

Vaadin ERP dirancang untuk mengatasi tantangan tersebut dengan mengadopsi paradigma **Metadata-Driven Architecture** dan **Low-Code Platform**. Sistem ini memisahkan antara *engine antarmuka* (*UI Render Engine*) dengan *definisi bisnis* (*Metadata Schema*). Setiap form, tabel, kolom, relasi master-detail, dan aturan validasi didefinisikan sebagai konfigurasi data di dalam tabel database (terutama di skema *dynamic*).

1.2 Keunggulan Arsitektur Metadata-Driven

- Zero Hard-Coded UI:** Tampilan form tidak ditulis secara statis. Ketika administrator menambahkan kolom baru melalui *Form Builder* di antarmuka, aplikasi secara otomatis merender komponen UI baru tersebut tanpa perlu *restart* atau kompilasi kode.

- **Konsistensi UI/UX Enterprise:** Seluruh aplikasi menggunakan komponen standar yang telah dioptimalkan (seperti `StandardActionToolBar`, `BandboxField`, dan `LovComboBox`), menjamin pengalaman pengguna yang seragam di ratusan form bisnis.
- **Skalabilitas Tinggi:** Penambahan modul bisnis baru (seperti Modul Inventaris, Pembelian, Keuangan, atau SDM) dapat dilakukan sepenuhnya melalui konfigurasi visual oleh Analis Bisnis.

1.3 Komponen Core & Alur Kerja Data (Data Flow)

Alur kerja sistem bermula ketika pengguna mengakses suatu menu transaksi. Engine antarmuka (seperti `GenericFormView` atau `GenericMasterDetailFormView`) akan melakukan query ke tabel `dynamic.form_header` dan `dynamic.form_detail`. Berdasarkan metadata yang diperoleh, kelas `ComponentFactory` menghasilkan secara dinamis komponen Vaadin yang sesuai (misalnya mengubah tipe kolom `DATE` menjadi kalender interaktif, atau `FILE` menjadi area unduh/unggah dokumen).

BAB 2: PANDUAN PENGOPERASIAN PENGGUNA (USER MANUAL)

2.1 Akses Sistem, Login, & Dashboard Portal

Akses sistem dilakukan melalui web browser standar (Google Chrome, Microsoft Edge, atau Mozilla Firefox) tanpa perlu menginstal aplikasi tambahan di komputer pengguna.

1. Buka browser dan masukkan alamat URL server ERP (contoh lokal: `http://localhost:8080`).
2. Pada halaman **Login** (`LoginView.java``), masukkan *Username* dan *Password* yang telah diberikan oleh Administrator Sistem.
3. Setelah otentikasi berhasil, Anda akan diarahkan ke halaman **Dashboard Portal** (`PortalView.java``). Menu navigasi di sisi kiri menampilkan modul-modul bisnis sesuai dengan hak akses (Role) yang Anda miliki.

2.2 Pengelolaan Data Master (Form Tunggal / Single Form)

Form Tunggal (`GenericFormView.java``) digunakan untuk mengelola data master referensi tunggal seperti Daftar Pelanggan, Daftar Supplier, Data Barang, atau Daftar Gudang.

- **Mencari & Menyaring Data:** Pada bagian atas tabel, terdapat baris pencarian multi-kolom. Ketikkan kata kunci pada kolom yang diinginkan (misal: Nama Supplier atau Kode Gudang), dan tabel akan menyaring data secara otomatis.
- **Menambah Data Baru:** Klik tombol **[+ New / Tambah]** pada toolbar atas (`StandardActionToolbar``). Form input akan terbuka di panel bawah atau samping.
- **Menyimpan Data:** Isi seluruh kolom yang wajib (ditandai dengan bintang merah). Klik tombol **[Save / Simpan]**. Sistem akan melakukan validasi metadata sebelum menyimpan data ke database.
- **Mengubah & Menghapus:** Klik baris data pada tabel untuk melihat detailnya. Lakukan perubahan teks dan klik Simpan, atau klik tombol **[Delete / Hapus]** untuk menghapus record.

2.3 Transaksi Kompleks Master-Detail (Header-Detail)

Form Master-Detail (`GenericMasterDetailView.java`) dirancang untuk menangani transaksi berjenjang yang memiliki dokumen induk (Header) dan daftar rincian barang/jasa (Detail), seperti *Purchase Order (PO)*, *Faktur Penjualan*, *Surat Jalan*, atau *Jurnal Akuntansi*.

A. Pengelolaan Bagian Header

Bagian atas form menampilkan data umum dokumen (seperti Nomor Dokumen, Tanggal Transaksi, Pemasok, Mata Uang, dan Keterangan). Data ini bertindak sebagai induk (Master) bagi seluruh baris rincian di bawahnya.

B. Pengelolaan Subform Grid (Detail Items)

Bagian bawah form adalah grid interaktif yang dikelola oleh komponen khusus `SubformGridField.java`.

- **Menambah Baris Rincian:** Klik tombol [\[+ Add Detail / Tambah Baris\]](#) pada sudut grid subform. Baris baru akan ditambahkan dan siap diisi.

Fitur Auto-Populate Dinamis: Sistem dilengkapi dengan logika pengisian otomatis yang sangat pintar:

- **Saat Memilih Barang/Item:** Ketika Anda memilih Kode Barang pada kolom grid detail menggunakan ComboBox atau Bandbox, sistem secara otomatis mengambil spesifikasi barang dari tabel master referensi (`mdlookup`) dan langsung mengisi kolom *Nama Barang*, *Satuan (UOM)*, dan *Harga Satuan* di baris yang sama.
- **Saat Event OnLoad:** Ketika Anda membuka dokumen existing, seluruh baris detail secara otomatis membaca parameter dari Header (misalnya Kode Produk Utama atau Kategori Pengadaan) untuk melakukan filter data yang relevan.
- **Menghapus Baris Detail:** Klik ikon tempat sampah pada baris rincian yang ingin dibatalkan. Baris tersebut akan dihapus dari grid dan akan dieksekusi penghapusannya di database saat Anda mengklik tombol Simpan utama.

2.4 Pencarian Data Referensi Lanjutan (LOV & Bandbox)

Untuk memastikan validitas data, input yang merujuk pada data lain menggunakan widget pemilih referensi (List of Values / LOV):

Widget Komponen	Karakteristik & Penggunaan Terbaik	Cara Pengoperasian
LOV ComboBox (<code>LovComboBox.java</code>)	Sangat cocok untuk daftar referensi dengan jumlah pilihan terbatas (< 200 baris), seperti Satuan (UOM), Status Dokumen, atau Tipe Akun.	Klik dropdown atau ketik langsung huruf awal kata kunci. Sistem akan menyaring daftar pilihan secara <i>auto-complete</i> .
Bandbox Grid Picker (<code>BandboxField.java</code>)	Dirancang khusus untuk dataset masif (ribuan hingga jutaan baris) seperti Master Barang, Daftar Suku Cadang (Sparepart), atau Pelanggan.	Klik ikon kaca pembesar di samping kotak teks. Jendela popup (<code>DynamicPickerPopupDialog</code>) akan muncul, menampilkan tabel lengkap dengan kolom filter dan pagination halaman. Pilih baris yang diinginkan dan klik <i>Pilih</i> .

2.5 Pengunggahan Dokumen & Gambar (Multi-File Upload)

Sistem mendukung pelampiran dokumen digital (seperti Scan Faktur Pajak, Foto Bukti Pengiriman, Kontrak PDF, atau Gambar Teknis Barang) melalui komponen `FileUploadField.java`.

- **Cara Unggah:** Tarik dan lepas (*drag-and-drop*) file dari Windows Explorer ke area kotak upload, atau klik tombol "**Browse**". Anda dapat memilih beberapa file sekaligus jika form mengizinkan multi-file.
- **Keamanan & Penamaan File Otomatis:** Anda tidak perlu khawatir nama file bentrok dengan pengguna lain. Sistem otomatis menyaring nama file Anda menjadi huruf kecil dan menambahkan awalan 8 karakter UUID acak (contoh: `e8b4f1a2_scan_faktur_2026.pdf`).
- **Melihat Pratinjau & Unduh:** File yang selesai diunggah akan muncul sebagai daftar ikon/tag. Klik pada nama file untuk melihat pratinjau dokumen atau mengunduhnya kembali.

2.6 Pelaporan Dinamis & Jejak Audit (Audit Trail)

- **Report Viewer (`ReportViewerView.java`)**: Untuk melihat laporan operasional, pilih menu Laporan. Tentukan parameter filter (misal: Tanggal Awal dan Akhir, atau Gudang), lalu klik *Tampilkan*. Anda juga dapat mengekspor laporan ke format Microsoft Excel (.xlsx) atau PDF.
- **Audit Log (`FieldAuditLogView.java`)**: Sistem merekam setiap aktivitas modifikasi data demi keamanan dan kepatuhan (compliance). Pada menu Audit Log, Anda dapat melacak siapa pengguna yang mengubah data, tanggal & jam eksekusi, nilai sebelum diubah (*Old Value*), dan nilai setelah diubah (*New Value*).

BAB 3: PANDUAN ADMINISTRATOR & LOW-CODE STUDIO

Modul **Low-Code Studio** adalah pusat kendali bagi Administrator Sistem dan Analis Bisnis untuk membangun, menyesuaikan, dan memelihara modul-modul ERP tanpa ketergantungan pada tim programmer.

3.1 Table Designer: Perancangan Skema Database Visual

Akses `TableDesignerView.java` untuk mengelola tabel di database PostgreSQL langsung dari antarmuka grafis:

- **Membuat Tabel Baru:** Tentukan nama tabel (di skema `dynamic` atau `public`), lalu tambahkan daftar kolom beserta spesifikasi tipe datanya (seperti `VARCHAR(100)`, `INTEGER`, `NUMERIC(18, 2)`, `TIMESTAMP`, `BOOLEAN`, atau `TEXT`).
- **Modifikasi Tabel Existing:** Anda dapat menambahkan kolom baru pada tabel transaksi yang sudah berjalan tanpa merusak data yang ada. Sistem akan mengeksekusi perintah DDL (`ALTER TABLE`) secara aman di latar belakang.

3.2 Form Builder: Perancang Form Tanpa Coding

Akses `FormBuilderView.java` untuk merancang antarmuka form transaksi secara visual:

1. **Ikatan Tabel (Table Binding):** Pilih tabel utama database yang akan dijadikan dasar penyimpanan data form.

Konfigurasi Kolom Form: Untuk setiap kolom, Anda dapat mengatur:

- *Label UI:* Nama teks yang muncul di layar (misal: "Tanggal Order" untuk kolom `order_dt`).
- *Sequence / Urutan:* Angka urutan tata letak kolom pada form.
- *Widget Component:* Pilih tipe kontrol UI (`TextBox`, `ComboBox`, `Bandbox`, `DatePicker`, `FileUpload`, atau `NumberField`).
- *Validasi:* Cendang opsi **Required** jika kolom wajib diisi, atau **Read-Only** jika kolom hanya untuk ditampilkan.

3. **Konfigurasi Relasi Master-Detail:** Hubungkan Form Header dengan Form Detail melalui penetapan kolom Foreign Key, sehingga sistem otomatis merender antarmuka transaksi berjenjang beserta subform grid-nya.

3.3 LOV & Lookup Builder: Pengelolaan Kategori dan Referensi

Akses `LovBuilderView.java` untuk mengelola kamus data referensi global yang tersimpan di dalam tabel `dynamic.mhlookup` (Master Header Category) dan `dynamic.mdlookup` (Master Detail Reference).

- **Membuat Kategori Baru:** Daftarkan kode kategori master (contoh: `MAS0020` untuk Kategori Pengadaan / Procurement, atau `MAS0003` untuk Satuan UOM).
- **Menambah Rincian Referensi:** Masukkan item-item referensi di bawah kategori tersebut. Item yang Anda masukkan di sini akan langsung muncul sebagai pilihan dropdown pada form transaksi yang menggunakan LOV ComboBox atau Bandbox.

3.4 Report Builder & Viewer: Pembuatan Laporan Custom

Akses `ReportBuilderView.java` untuk merancang laporan bisnis baru:

- Masukkan query SQL atau pilih Database View sebagai sumber data utama laporan.
- Tentukan parameter filter dinamis yang dapat diisi oleh pengguna (misal: Filter Rentang Tanggal, Filter Kode Supplier).
- Atur kolom-kolom yang akan ditampilkan pada tabel hasil laporan beserta format angkanya (misal: format mata uang Rupiah atau desimal).

3.5 User Authority Admin: Manajemen Hak Akses & Keamanan

Akses `UserAuthorityAdminView.java` untuk mengatur keamanan sistem berdasarkan prinsip *Role-Based Access Control (RBAC)*:

- **Manajemen Role:** Buat grup peran pengguna sesuai struktur organisasi perusahaan (misal: *Super Admin, Manajer Pembelian, Staff Gudang, Staff Akuntansi*).

Matriks Hak Akses Form & Menu: Atur hak akses sangat mendetail (Granular Security) untuk setiap Role pada setiap form:

- **Read (R):** Hak untuk melihat dan membuka form/laporan.
- **Create (C):** Hak untuk menekan tombol Tambah dan menyimpan data baru.
- **Update (U):** Hak untuk mengubah data yang sudah ada.
- **Delete (D):** Hak untuk menghapus data dari database.

BAB 4: ARSIKTETUR TEKNIS & ATURAN WAJIB PENGEMBANG (DEVELOPER GUIDE)

Bab ini ditujukan bagi pengembang sistem (Developer) dan AI Engineering Assistant yang melakukan modifikasi kode sumber pada repositori `vaadinerp`. Seluruh pengembang **WAJIB** mematuhi standar arsitektur yang telah ditetapkan dalam dokumen aturan pengembang (`AGENTS.md`).

ATURAN KRUSIAL PROYEK: ARSITEKTUR FILE UPLOAD & STORAGE

Setiap fitur yang melibatkan pengunggahan file, gambar, atau penyimpanan dokumen wajib mematuhi 4 pilar arsitektur penyimpanan di bawah ini tanpa pengecualian!

4.1 Standar Wajib Arsitektur File Upload & Penyimpanan Dokumen

Pilar 1: Separate Storage & DB Reference (Anti-BLOB / BYTEA)

JANGAN PERNAH menyimpan data binary file (tipe data `BLOB` atau `BYTEA`) secara langsung ke dalam tabel database PostgreSQL.

- Simpan file fisik di disk sistem operasi server (pada folder standar `./uploads` atau folder yang dikonfigurasi melalui properti aplikasi `app.upload.dir` di `application.properties`).
- Database **hanya menyimpan nama file** atau *relative path*-nya di dalam kolom bertipe data `TEXT` atau `VARCHAR`.
- Alasan Arsitektur:* Menyimpan binary di database menyebabkan ukuran database membengkak dengan cepat, memperlambat proses pencadangan (backup/restore), dan membebani memori RAM server saat melakukan query tabel.

Pilar 2: Sanitasi Nama File & UUID 8-Karakter Prefix (Anti-Overwrite)

Semua file yang diunggah oleh pengguna wajib melalui proses sanitasi dan modifikasi nama sebelum disimpan ke disk:

1. **Sanitasi String:** Nama file asli diubah menjadi huruf kecil (*lowercase*). Seluruh spasi dan karakter khusus diganti dengan karakter garis bawah (*underscore* `_`).
2. **UUID Prefix:** Wajib menambahkan awalan 8 karakter acak UUID yang diikuti tanda garis bawah di depan nama file yang telah disanitasi.

Contoh: "Faktur Pajak PT GMN 2026.pdf" →
"a1b2c3d4_faktur_pajak_pt_gmn_2026.pdf"

Alasan Arsitektur: Mencegah penimpaan file secara tidak sengaja (*overwrite*) jika dua pengguna mengunggah file dengan nama yang sama. Selain itu, menjamin kompatibilitas 100% antara server Windows (yang bersifat *case-insensitive*) dan server Linux produksi (yang bersifat *case-sensitive*).

Pilar 3: Multi-File Comma-Delimited String Storage

Untuk field atau komponen antarmuka yang mendukung pengunggahan banyak file sekaligus (Multi-File Upload), simpan daftar nama file fisik tersebut sebagai **string terpisah koma (comma-delimited string)** pada satu kolom bertipe data `TEXT` di PostgreSQL.

Contoh	isi	kolom	DB:
"a1b2c3d4_doc1.pdf, f9e8d7c6_foto_barang.jpg, b5a4c3d2_kontrak.pdf"			

Pilar 4: Cascade Physical File Deletion (Anti-Disk Leak)

Saat melakukan penghapusan record data dari database melalui service backend (terutama di kelas `DynamicDataService.deleteData`), baik pada penghapusan data tunggal, data master-detail, maupun baris subform, sistem **WAJIB memeriksa kolom file dan menghapus file fisik terkait di disk server**.

Alasan Arsitektur: Jika record di database dihapus tanpa menghapus file fisiknya di folder `./uploads`, maka server akan mengalami kebocoran kapasitas penyimpanan (*disk space leak*) akibat penumpukan file yatim (*orphan files*) yang tidak lagi terpakai.

4.2 Skema Kamus Referensi Database (`dynamic.mhlookup` & `dynamic.mdlookup`)

Sistem Vaadin ERP memusatkan seluruh kamus referensi pilihan (LOV, Status, Kategori) pada dua tabel utama di dalam skema `dynamic`:

A. Tabel Master Header (`dynamic.mhlookup`)

Menyimpan kelompok atau kategori referensi utama.

Nama Kolom	Tipe Data PostgreSQL	Keterangan & Spesifikasi
id	BIGSERIAL / BIGINT	Primary Key.
category_code	VARCHAR(50)	Kode unik kategori (Wajib batas maksimal 50 karakter). Contoh: MAS0001, MAS0020.
category_name	VARCHAR(255)	Nama deskriptif kategori. Contoh: Master : Procurement Category.
status	BOOLEAN	Status aktif kategori (default: true).

B. Tabel Master Detail (^dynamic.mdlookup`)

Menyimpan daftar rincian pilihan untuk setiap kategori master.

Nama Kolom	Tipe Data PostgreSQL	Keterangan & Spesifikasi
id	BIGSERIAL / BIGINT	Primary Key.
global_category_id	BIGINT	Foreign Key mengarah ke dynamic.mhlookup(id).
code	VARCHAR(50)	Kode unik rincian item (Wajib batas maksimal 50 karakter). Contoh: 3.13-001, 01. Raw Material.

<code>name / codetext</code>	<code>VARCHAR(255)</code>	Teks tampilan yang muncul di dropdown ComboBox/ComboBox. Contoh: <code>ADHESIVE</code> (Perekat/ Lem).
<code>description</code>	<code>VARCHAR(255)</code>	Keterangan tambahan atau catatan spesifikasi barang.

PERINGATAN BATAS TIPE DATA (VALUE TOO LONG MITIGATION)

Kolom `code` pada tabel `mdlookup` dibatasi maksimal **50 karakter**. Saat membuat skrip migrasi atau import data otomatis (seperti skrip Python generator dari TSV/Excel), pengembang **wajib melakukan pemotongan string otomatis (truncation: `val[:50]`)** sebelum mengeksekusi perintah INSERT agar tidak terjadi error SQL *value too long for type character varying(50)*.

4.3 Arsitektur Backend & Database Service

- **`DynamicDataService.java`:** Service utama berbasis Spring Service yang bertugas mengeksekusi operasi CRUD (Create, Read, Update, Delete) dinamis menggunakan MyBatis / JDBC Template. Service ini otomatis menyusun query SQL berdasarkan definisi kolom di tabel metadata.
- **`DynamicFormService.java`:** Service yang bertanggung jawab mengambil definisi struktur form (``form_header`` dan ``form_detail``) serta mengelola eksekusi transaksi master-detail bersarang (nested transaction).
- **Konfigurasi Multi-Schema:** Pada file `src/main/resources/application.properties`, koneksi PostgreSQL wajib dikonfigurasi untuk memindai skema public dan dynamic:
`spring.datasource.url=jdbc:postgresql://localhost:5432/vaadinerp?currentSchema=public,dy`

BAB 5: LAPORAN HASIL IMPLEMENTASI & VERIFIKASI DATA MASTER LOOKUP

Sebagai bagian dari verifikasi implementasi sistem, tim telah berhasil melakukan proses pembersihan, pemetaan, dan injeksi data referensi skala besar (Enterprise Lookup Dictionary) yang diberikan oleh pengguna ke dalam tabel database produksi.

5.1 Statistik Integrasi Data Master & Detail

Proses import dieksekusi menggunakan skrip generator Python yang menghasilkan perintah SQL atomik dalam transaksi (`BEGIN ... COMMIT`) dengan pengecekan idempotent (`WHERE NOT EXISTS`). Hasil verifikasi database menunjukkan:

- Total Kategori Master (`dynamic.mhlookup``): 22 Kategori Aktif** (Terdiri dari 4 kategori bawaan sistem dan 18 kategori enterprise baru yang diimpor dari dataset pengguna).
- Total Rincian Detail (`dynamic.mdlookup``): 931 Baris Rincian Aktif** yang terhubung sempurna secara relasional melalui Foreign Key `global_category_id`.

5.2 Daftar Kategori Master Referensi Aktif (`dynamic.mhlookup``)

Berikut adalah daftar lengkap 22 kategori referensi yang kini aktif dan siap digunakan pada seluruh form transaksi di Vaadin ERP:

Kode Kategori (<code>category_code</code>)	Nama Kategori (<code>category_name</code>)	Status	Peran dalam Sistem ERP
MAS0001	Master : Account Type	Aktif	Klasifikasi Tipe Akun Akuntansi (Debit/Kredit)
MAS0002	Master : Item Type	Aktif	Klasifikasi Barang (Stock vs Service/Jasa)

MAS0003	Master : UOM	Aktif	Satuan Ukuran Barang (Pcs, Kg, Meter, Liter, dll)
MAS0004	Master : Business Partner Type	Aktif	Tipe Mitra Bisnis (Customer, Supplier, Vendor)
MAS0006	Master : Down Payment Type	Aktif	Jenis Uang Muka Transaksi
MAS0007	Master : Delivery Mode	Aktif	Metode Pengiriman Barang
MAS0008	Master : Delivery Term	Aktif	Syarat Pengiriman (FOB, CIF, Ex-Works)
MAS0009	Master : Purchase Type	Aktif	Jenis Pembelian / Pengadaan
MAS0010	Master : Purchase Payment Type	Aktif	Metode Pembayaran Pembelian (Cash, Credit, CBD)
MAS0011	Master : Sales Type	Aktif	Jenis Penjualan
MAS0012	Master : Sales Payment Type	Aktif	Metode Pembayaran Penjualan
MAS0013	Master : Element	Aktif	Elemen Komponen Biaya / Produksi

MAS0014	Master : Item Parent Group	Aktif	Grup Induk Barang Inventaris
MAS0015	Master : Item Group	Aktif	Sub-Grup Klasifikasi Barang Inventaris
MAS0016	Master : Resource Group	Aktif	Kelompok Sumber Daya Mesin / Tenaga Kerja
MAS0017	Master : Site	Aktif	Lokasi Pabrik / Cabang Perusahaan
MAS0018	Master : Resource	Aktif	Detail Sumber Daya Produksi
MAS0020	Master : Procurement Category	Aktif	Kategori Pengadaan Barang & Jasa (700+ Rincian)

5.3 Sampel Data Rincian Kategori Pengadaan (`MAS0020`)

Kategori MAS0020 (Procurement Category) merupakan dataset referensi terbesar yang telah berhasil diintegrasikan, mencakup lebih dari 700 rincian barang teknis dan operasional pabrik. Berikut adalah sampel representatif dari rincian yang tersimpan di `dynamic.mdlookup`:

Kode Rincian (code)	Nama Referensi / Teks Tampilan (name)	Kelompok Barang / Aplikasi
01. Raw Material	01. Raw Material	Bahan Baku Utama Produksi

1.01-011.01	RM Foundry - Sand - Chromite	Bahan Baku Foundry (Pasir Kromit)
1.02-001.01	RM Fastener - AS Steel Bar 42CRMO - Bolt	Bahan Baku Fastener (Baja 42CrMo Bolt)
1.03-001	RM Rubber - Polymer	Bahan Baku Karet (Polimer)
2.01-001.01	Additive - Alloy - Ferro Chromium	Bahan Aditif Logam (Ferro Chromium)
3.12-001	PLAT	Bahan Bangunan & Konstruksi (Plat Besi)
3.13-001	ADHESIVE (Perekat/ Lem)	Barang Consumable Pabrik (Lem/Perekat)
3.13-005	WELDING WIRE	Barang Consumable (Kawat Las)
3.14-001	BREAKER	Komponen Kelistrikan / Electrical (Breaker)
3.14-035	PLC	Komponen Otomasi Industri (PLC)
3.18-002	LAPTOP	Perangkat IT & Komputer (Laptop/PC)
3.24-001	HELMET	Perlengkapan Keselamatan / APD (Helm Safety)
3.25-012	BEARING	Suku Cadang Mesin / Sparepart (Bearing)

3.28-004	GRINDER (GERINDA)	Peralatan Kerja / Tools (Mesin Gerinda)
----------	-------------------	--

5.4 Panduan Mitigasi Pemotongan Teks (Chat Truncation Mitigation)

Selama proses implementasi, tim mengidentifikasi fenomena teknis di mana pengunggahan teks masif (ribuan baris TSV) melalui kolom chat web antarmuka mengalami pemotongan otomatis oleh browser atau server batas karakter (*message truncation*).

Prosedur Standar (SOP) untuk Import Data Master Skala Besar Berikutnya:

- Hindari Copy-Paste Langsung via Chat untuk Data > 200 Baris:** Jika dataset memiliki lebih dari 200 baris atau melebihi 20.000 karakter, jangan menyalin langsung ke kolom pesan chat.
- Gunakan Metode Upload File Proyek:** Simpan data referensi dalam format file standar (seperti `.tsv`, `.csv`, atau `.xlsx`). Letakkan file tersebut di dalam folder proyek (misalnya di `root d:\PROJECT GMN\vaadinerp\` atau folder `./uploads`).
- Instruksikan Pembacaan File Langsung:** Beritahu nama file kepada AI Assistant atau sistem impor. Sistem akan menggunakan tool pembacaan file sistem (*file system reader*) yang mampu memproses ratusan ribu baris data secara utuh 100% tanpa risiko terpotong.